

Manual De Desenvolvimento Do LabDessem Rust

1. Objetivo deste manual

Este manual foi escrito para servir como guia prático de manutenção e evolução do projeto `labdessem_rust`, especialmente para quem ainda não tem muita familiaridade com Rust.

O foco aqui não é explicar teoria de otimização em abstrato, mas sim mostrar:

- como o código está organizado;
- por onde os dados entram;
- como o modelo matemático é montado;
- como adicionar uma nova variável;
- como adicionar uma nova restrição;
- como alterar a função objetivo;
- como incluir novas leituras de arquivos de entrada;
- como expor os resultados em arquivos de saída;
- quais boas práticas seguir para não quebrar a estrutura existente.

Este texto foi pensado para ser usado como um manual de trabalho. A ideia é que você consiga abrir o arquivo, localizar a seção relevante e seguir um roteiro seguro.

2. Visão geral da arquitetura

O projeto está dividido em crates com responsabilidades bem definidas:

``crates/labdessem-core``

Camada de domínio. Define as estruturas centrais do problema:

- `System`
- `StudyHorizon`
- `Submarket`
- `Bus`
- `Branch`
- `ThermalPlant`, `ThermalUnit`
- `HydroPlant`, `HydroGroup`, `HydroUnit`, `Reservoir`

- `PumpingPlant`
- `WindPlant`, `SolarPlant`
- limites operativos, custos residuais, ids tipados

Também concentra validações estruturais do sistema.

Arquivo central:

- [system.rs](#)

Arquivos importantes:

- [thermal.rs](#)
- [hydro.rs](#)
- [renewable.rs](#)
- [ids.rs](#)

`crates/labdessem-io`

Camada de leitura e tradução dos arquivos de entrada para `System`.

Responsabilidades:

- ler `study_config.json`;
- ler arquivos `CAD/*.csv` e `OPER/*.csv`;
- converter unidades;
- montar horizonte;
- construir objetos do domínio;
- validar coerência básica dos dados lidos.

Arquivo central:

- [study.rs](#)

`crates/labdessem-model`

Camada de formulação matemática.

Responsabilidades:

- criar índices internos;
- declarar variáveis;
- montar restrições lineares;
- montar a função objetivo.

Arquivos centrais:

- [lib.rs](#)
- [indexing.rs](#)
- [variables.rs](#)
- [constraints.rs](#)
- [objective.rs](#)

`crates/labdessem-solver`

Camada de interface com o solver `good_lp`.

Responsabilidades:

- transformar a estrutura do `Model` em variáveis do solver;
- converter restrições e objetivo para expressões lineares;
- resolver LP ou MILP;
- coletar valores primais;
- coletar duais em problemas LP.

Arquivo central:

- [lib.rs](#)

`crates/labdessem-simulation`

Camada de orquestração do processo de resolução e geração de resultados.

Responsabilidades:

- escolher a estratégia de execução;
- rodar LP, MILP, LP-FIXED e LP-CALC-CMO;
- construir cortes de rede quando aplicável;
- tratar folgas de inviabilidade;
- consolidar saídas CSV.

Arquivo central:

- [lib.rs](#)

`crates/labdessem-cli`

Ponto de entrada do programa.

Responsabilidades:

- ler a configuração;
- chamar a leitura do caso;

- disparar a simulação;
- escrever resultados.

Arquivo:

- [main.rs](#)

3. Fluxo completo do programa

O fluxo principal é:

1. O `labdessem-cli` lê o `study_config.json`.
2. O `labdessem-io` lê todos os arquivos do caso e monta um `System`.
3. O `system` é validado.
4. O `labdessem-model` transforma o `System` em:
 - `Indexing`
 - `Variables`
 - `ConstraintSet`
 - `Objective`
5. O `labdessem-solver` resolve o problema.
6. O `labdessem-simulation` usa a solução para:
 - rodar iterações adicionais, quando necessário;
 - calcular fluxos;
 - calcular duais;
 - gerar arquivos de saída.

Se você quiser implementar algo novo, quase sempre a modificação passa por uma ou mais das seguintes camadas:

- `core`: quando surge um novo dado estrutural;
- `io`: quando surge um novo arquivo, nova coluna ou nova conversão;
- `model`: quando surge variável, restrição ou termo da FOB;
- `simulation`: quando surge nova lógica de execução ou nova saída.

4. Como pensar a estrutura do cÃ³digo

Uma regra prÃ¡tica muito importante:

- `core` representa o mundo do problema;
- `io` traduz o mundo externo para o `core`;
- `model` traduz o `core` para o modelo matemÃ¡tico;
- `solver` resolve;
- `simulation` interpreta o resultado e gera relatÃ³rios.

Em outras palavras:

- nÃ£o coloque lÃ³gica de leitura de CSV em `model`;
- nÃ£o coloque regra de solver em `io`;
- nÃ£o coloque nome de arquivo de saÃda em `core`;
- nÃ£o coloque regra de negÃcio estrutural dentro do CLI.

Isso ajuda muito a manter o projeto compreensÃvel.

5. Tipos fundamentais que vocÃª precisa entender

`System`

Ã‰ o objeto mestre do caso.

Ele contÃ©m:

- horizonte;
- submercados;
- barras;
- linhas;
- tÃ©rmicas;
- hidrelÃ©tricas;
- renovÃ¡veis;
- elevatÃ³rias;
- limites operativos;
- custos residuais;
- flags de UC.

Se uma nova modelagem precisa existir no problema, normalmente alguma parte dela precisa aparecer em `System`.

`Indexing`

Como as plantas podem ter várias unidades e o horizonte tem muitos períodos, o modelo trabalha com vetores lineares.

O `Indexing` guarda o mapeamento entre:

- posição no vetor;
- planta;
- unidade;
- grupo;
- submercado;
- período.

Sempre que você criar uma variável indexada por unidade, planta ou submercado, confira se o `Indexing` já possui uma entrada adequada. Se não possuir, ele pode precisar ser ampliado.

`Variables`

É o catálogo de variáveis matemáticas do modelo.

Cada variável tem:

- `name`
- `lower_bound`
- `upper_bound`
- `domain`
- `fixed_value`

As variáveis são armazenadas em vetores por família:

- `thermal_generation`
- `hydro_generation`
- `hydro_turbining`
- `hydro_spillage`
- `hydro_volume`
- `thermal_commitment`
- `thermal_startup`
- `thermal_shutdown`
- etc.

`LinearConstraint`

Cada restrição é guardada como:

- `name`
- `terms: Vec<LinearTerm>`
- `sense`
- `rhs`

Um `LinearTerm` tem:

- `variable`
- `coefficient`

Ou seja, o modelo é explicitamente montado como Álgebra linear.

`Objective`

A função objetivo é uma lista de `ObjectiveTerm`, cada um com:

- nome da variável;
- coeficiente.

6. Convenções importantes do projeto

6.1 Convenção de nomes

Os nomes das variáveis seguem uma estrutura legível. Exemplos:

- `thermal_generation[p=Termorio,u=6,t=1]`
- `hydro_generation[p=CAMARGOS,g=1,u=2,t=3]`
- `hydro_volume[p=CAMARGOS,t=0]`

Ao criar variáveis novas, mantenha o padrão:

- prefixo descritivo;
- índices nomeados;
- período explícito quando aplicável.

Isso ajuda muito:

- na leitura de logs;
- na depuração;
- nos testes;
- na interpretação de soluções.

6.2 Unidade física

Antes de implementar qualquer coisa, defina com clareza:

- em que unidade o dado entra;
- em que unidade a variável vive internamente;
- em que unidade a restrição foi escrita;
- em que unidade a saída será impressa.

Grande parte dos erros em modelos hidroenergéticos nasce de inconsistência dimensional.

6.3 Uma única unidade interna por conceito

Se possível, escolha uma unidade interna única para cada conceito.

Exemplos do projeto atual:

- turbinamento interno: hm^3 por período
- vertimento interno: hm^3 por período
- afluência lida em m^3/s , mas convertida para hm^3 por período
- geração: MW
- custo térmico variável: $\text{R}\$/\text{MWh}$
- penalidade hidráulica de turbinamento/vertimento: $\text{R}\$/\text{hm}^3$

Não fique alternando unidade interna no meio da modelagem.

6.4 Validação cedo

Se um dado inválido puder ser detectado na leitura, detecte na leitura.

Exemplos:

- nome de usina inexistente;
- limite operacional aplicado a variável incompatível com o tipo da usina;
- rampa inicial marcada sem trajetória;
- volume inicial fora dos limites;
- submercado inexistente.

Isso evita erros silenciosos mais adiante.

7. Como adicionar uma nova variável

Esta é uma das tarefas mais comuns.

7.1 Perguntas que você deve responder antes

Antes de codar, responda:

1. Essa variável representa o quê?
2. Ela pertence a qual entidade?
 - usina
 - unidade
 - grupo
 - submercado
 - barra
 - linha
3. Ela é indexada no tempo?
4. Ela é contínua ou binária?
5. Quais são os limites inferior e superior?
6. Ela precisa aparecer em arquivo de saída?
7. Ela participa de restrições?
8. Ela participa da função objetivo?
9. Ela precisa de leitura de dado de entrada?

7.2 Passos práticos

Passo 1. Criar ou adaptar o dado estrutural em `core`

Se a nova variável depende de um novo parâmetro, acrescente esse parâmetro na estrutura adequada em `labdessem-core`.

Exemplos:

- um novo limite técnico de unidade térmica;
- um novo custo hidráulico;
- uma nova vazão máxima;
- uma nova relação entre plantas.

Onde olhar:

- [thermal.rs](#)
- [hydro.rs](#)
- [system.rs](#)

Depois, ajuste a validação da estrutura.

Passo 2. Ler o dado em `labdessem-io`

Se a variável precisa de informação nova dos arquivos:

- adicione coluna na `struct` que representa a linha CSV;
- use `#[serde(rename = "...")]`;
- converta a unidade, se necessário;
- carregue o valor para dentro do objeto do `core`.

Onde olhar:

- [study.rs](#)

Exemplo mental:

```
[serde(rename = "NovaColuna")]
novo_campo: f64;
```

Passo 3. Verificar se o `Indexing` já suporta a entidade

Se a variável não:

- por unidade térmica: provavelmente `thermal_unit_entries` basta;
- por unidade hidráulica: provavelmente `hydro_unit_entries` basta;
- por usina hidráulica: provavelmente `hydro_plant_entries` basta;
- por usina elevatória: provavelmente `pumping_plant_entries` basta;
- por submercado: use `system.submarkets`;
- por par de submercados e período: use `interchange_entries`.

Se a nova indexação não existir, amplie [indexing.rs](#).

Passo 4. Declarar a variável em `Variables`

Em [variables.rs](#):

1. adicione um novo campo no `struct Variables`;
2. construa o vetor em `Variables::for_system`;
3. defina nome, bounds, domínio e valor fixo;
4. adicione a variável em `collect_variables` no solver.

Exemplo conceitual:

```
pub new_variable: Vec<Variable>,
```

e depois:

```
let new_variable = indexing
    .algum_indice
    .iter()
    .flat_map(|entry| {
        (0..horizon).map(move |period| Variable {
            name: format!("new_variable[p={},t={}]", ...),
            lower_bound: 0.0,
            upper_bound: None,
            domain: VariableDomain::Continuous,
            fixed_value: None,
        })
    })
    .collect();
```

Passo 5. Incluir a variável no solver

Em [labdessem-solver/src/lib.rs](#), a função `collect_variables` precisa conhecer a nova família.

Se você esquecer isso, o solver não verá a variável.

Passo 6. Usar a variável em restrições ou na FOB

Depois de criada, ela ainda não faz nada sozinha.

Você precisa:

- colocá-la em restrições, se for o caso;
- colocá-la no objetivo, se for o caso;
- talvez imprimi-la em saídas.

Passo 7. Adicionar teste

No momento, adicione um teste que verifique:

- que a variável foi criada;
- que nome e bounds estão corretos;
- que a contagem de variáveis mudou como esperado.

Os testes do `labdessem-model` são o lugar mais natural para isso.

8. Como adicionar uma nova restrição

8.1 Passo conceitual

Antes de implementar, escreva a restrição matematicamente.

Faça questão de deixar claro:

- qual o índice da restrição;
- quais variáveis entram;
- em que unidade está o lado esquerdo;
- em que unidade está o lado direito;
- se $=$, $\leq$ ou $\geq$.

Nunca implemente a restrição primeiro e entenda depois.

8.2 Onde colocar

As restrições lineares ficam em:

- [constraints.rs](#)

O padrão do projeto é:

1. criar uma função `build...constraints`;
2. fazer essa função retornar `Vec<LinearConstraint>`;
3. registrar a função dentro de `ConstraintSet::for_system`.

8.3 Modelo de implementação

Uma restrição típica é montada assim:

```
constraints.push(LinearConstraint {
    name: format!("nome da restricao[...]"),
    terms: vec![
        term(&variavel_1, coef_1),
        term(&variavel_2, coef_2),
    ],
    sense: ConstraintSense::LessOrEqual,
    rhs: valor_rhs,
});
```

8.4 Checklist para uma nova restrição

Quando for criar uma restrição, verifique:

1. Os coeficientes estão em unidade compatível?
2. O `rhs` está na mesma unidade do lado esquerdo?
3. Os índices estão corretos?
4. O nome da restrição identifica bem a entidade e o período?
5. A restrição deve existir em LP, MILP e LP-FIXED, ou só em alguns modos?
6. Ela depende de alguma flag, como `UCT`, `UCH` ou `rede`?
7. Ela precisa de folga em caso de inviabilidade?

8.5 Controle por modo de resolução

No projeto, algumas restrições só entram quando o modo é MILP ou LP com compromisso fixo.

Isso é controlado em `ConstraintSet::for_system`.

Exemplo:

- restrições de UC térmico só entram se:
 - o `solve_mode` for adequado;
 - e `system.thermal_unit_commitment_enabled` for verdadeiro.

Se sua nova restrição depende de uma lógica semelhante, siga o mesmo padrão.

8.6 Restrições com nomes auxiliares

Se quiser que a restrição seja mais fácil de encontrar depois, padronize o prefixo.

Exemplos existentes:

- `demand_balance[...]`
- `hydro_balance[...]`
- `hydro_fpha[...]`
- `thermal_min_up[...]`
- `thermal_ramp...`

Isso ajuda:

- a depurar;

- a calcular duais;
- a criar filtros em testes;
- a buscar a restrição no relatório.

9. Como modificar a função objetivo

A FOB fica em:

- [objective.rs](#)

O padrão é simples:

1. percorra os índices relevantes;
2. recupere a variável;
3. empurre um `ObjectiveTerm`.

Exemplo conceitual:

```
terms.push(term(variavel, coeficiente));
```

9.1 Sempre confira a unidade do coeficiente

Exemplos do projeto:

- geração térmica:
 - variável em MW
 - custo em R\$/MWh
 - precisa multiplicar pela duração do período em horas
- vertimento hidráulico:
 - variável em hm³ por período
 - custo em R\$/hm³
 - não precisa multiplicar por duração
- turbinamento hidráulico:
 - variável em hm³ por período
 - custo em R\$/hm³

- não precisa multiplicar por duração

Essa parte é crítica. Antes de editar a FOB, faça a análise dimensional explicitamente.

9.2 Cuidado com dupla contagem

Ao adicionar um termo de custo:

- confira se a variável já aparece em outro termo;
- confira se você está penalizando por usina e por unidade ao mesmo tempo sem querer;
- confira se uma penalidade de agregação já está implícita em outra parte.

9.3 Cuidado com termos não válidos em alguns modos

Custos de partida e desligamento, por exemplo, não fazem sentido quando há UC.

Se o termo depende de binária, condicione pela flag apropriada.

10. Como adicionar um novo dado de entrada

Há dois cenários.

10.1 Nova coluna em arquivo existente

Passos:

1. localizar a `struct` que representa a linha do CSV;
2. adicionar o campo com `serde(rename = "...")`;
3. usar o campo na montagem do objeto final;
4. validar o valor.

10.2 Novo arquivo

Passos:

1. criar uma `struct` para as linhas;
2. usar `read_csv(...)` ou função especializada;
3. chamar a leitura em `read_study_from_path_with_options`;

4. passar as linhas para a função construtora relevante;
5. transformar o dado em algo do `System`.

10.3 Quando criar função especializada

Use função especializada em vez de `read_csv` simples quando:

- o arquivo tem layout menos convencional;
- há cabeçalhos não triviais;
- há mais de uma tabela no mesmo arquivo;
- existe lógica de interpretação mais rica.

Exemplos no projeto:

- leitura de trajetórias;
- leitura de renováveis;
- leitura de FPHA.

11. Como adicionar um novo arquivo de saída

Os arquivos de saída ficam em:

- [labdessem-simulation/src/lib.rs](https://github.com/labdessem-simulation/src/lib.rs)

O padrão usado hoje é:

1. criar uma função `write..._csv`;
2. usar `csv_with_header(...)`;
3. montar as linhas com `format!`;
4. chamar `write_csv_file(...)`;
5. registrar a escrita dentro de `write_results_csvs`.

11.1 Boas práticas para saída

- descreva bem cada coluna no cabeçalho;
- não misture unidades numa mesma coluna;
- se houver mistura possível, adicione coluna `Unidade`;
- use nomes consistentes com o modelo;
- para valores agregados, deixe isso claro;
- para valores por usina, não repita como se fossem por unidade.

11.2 Armadilha comum

É muito fácil errar o número de placeholders no `format!`.

Depois de editar saldas:

- `rode cargo fmt;`
- `rode cargo check -p labdessem-cli.`

12. Como adicionar uma nova variável dual ou novo relatório dual

Os duais vêm do LP, não do MILP.

Se você quiser usar duais:

1. identifique qual restrição gera o dual desejado;
2. garanta que esse dual está sendo coletado em `SolveSummary`;
3. use o `LP-CALC-CMO` ou outro LP adequado para calcular o relatório;
4. recupere o dual pelo nome da restrição.

Exemplo do projeto:

- `PiDemanda`
- `PiBalHidr`

Se a restrição tiver nome ruim, esse tipo de relatório fica muito mais difícil. Por isso o nome da restrição importa.

13. Como depurar uma implementação nova

13.1 Estratégia recomendada

Siga esta ordem:

1. validar leitura;
2. validar estrutura em `System`;
3. validar criação da variável;

4. validar criação da restrição;

5. validar montagem da FOB;

6. validar saída.

Não tente depurar tudo ao mesmo tempo.

13.2 Ferramentas Úteis

- `cargo check -p labdessem-cli`
- `cargo test -p labdessem-model`
- `rg "texto"` para localizar nomes de variáveis/restrições

13.3 Depuração por inspeção textual

Este projeto favorece muito a depuração por nomes:

- procure a variável pelo nome;
- procure a restrição pelo prefixo;
- confira se a família foi adicionada em `collect_variables`;
- confira se a função construtora foi chamada em `ConstraintSet::for_system`.

14. Como adicionar testes de forma Útil

Os melhores testes para este projeto costumam ser:

14.1 Teste de montagem de variável

Verifica:

- quantidade de variáveis;
- bounds;
- domínio;
- nome.

14.2 Teste de montagem de restrição

Verifica:

- se a restrição existe;
- se o nome está correto;

- se os coeficientes estão corretos;
- se o `rhs` está correto;
- se a relação `<=`, `>=` ou `=` está certa.

14.3 Teste de leitura

Verifica:

- se o caso foi lido com sucesso;
- se campos novos foram populados;
- se erros de entrada são detectados.

14.4 Teste de regressão funcional

Quando uma mudança corrige um bug importante, escreva um teste pequeno que capture exatamente aquele bug.

15. Roteiro completo para implementar uma nova variável

Aqui vai um roteiro operacional resumido.

Exemplo genérico

Suponha que você queira criar uma nova variável contida em `x[p,t]`.

Etapa A. Domínio

1. Pergunte se `x` precisa de novos parâmetros no `System`.
2. Se sim, adicione em `core`.
3. Valide esses parâmetros.

Etapa B. Entrada

1. Adicione coluna ou arquivo em `io`.
2. Leia o dado.
3. Converta unidade.
4. Passe para o `System`.

Etapa C. Modelagem

1. Decida a indexação.
2. Adicione o campo em `Variables`.
3. Construa o vetor em `Variables::for_system`.
4. Adicione em `collect_variables`.

Etapa D. Restrições

1. Escreva a matemática.
2. Crie `build_x_constraints`.
3. Registre em `ConstraintSet::for_system`.

Etapa E. Objetivo

1. Se `x` participa da FOB, acrescente em `Objective::for_system`.
2. Confira a unidade do coeficiente.

Etapa F. Salva

1. Se `x` deve ser relatada, adicione em `write_..._csv`.
2. Descreva a coluna.

Etapa G. Testes

1. teste de leitura;
2. teste de variável;
3. teste de restrição;
4. teste de compilação.

16. Roteiro completo para implementar uma nova restrição

Exemplo genérico

Suponha uma restrição:

```
\[
\sum a_j x_j \le b
\]
```

Passos:

1. Liste todas as variáveis envolvidas.
 2. Descubra como localizar cada uma dentro de `Variables`.
 3. Descubra qual Índice usar em `Indexing`.
 4. Crie uma função `build_nome_da_restricao_constraints`.
 5. Monte `terms`.
 6. Escolha `ConstraintSense`.
 7. Defina `rhs`.
 8. Dê um nome rastreável.
 9. Registre a função no construtor do conjunto de restrições.
 10. Crie teste que verifique pelo menos um caso.
-

17. Roteiro completo para modificar a FOB

Passos:

1. escreva o novo termo matematicamente;
 2. faça a análise dimensional;
 3. localize a variável correta;
 4. defina o coeficiente correto;
 5. veja se depende do modo de resolução;
 6. implemente em `objective.rs`;
 7. valide se não houve dupla contagem;
 8. rode `cargo check`.
-

18. Como lidar com flags de configuração

As flags do `study_config.json` são lidas em `StudyConfig`.

Exemplos atuais:

- `rede`

- `UCT`
- `UCH`
- `TON_Residual`
- `opcao_execucao`

Se você criar uma nova funcionalidade opcional:

1. adicione o campo em `StudyConfig`;
2. repasse a flag para `read_study_from_path_with_options`, se necessário;
3. registre a decisão no ponto apropriado:
 - leitura
 - montagem de modelo
 - simulação
 - saída

Boas práticas:

- use `u8` como já está no padrão atual;
- interprete `0` como desligado e `1` como ligado;
- valide valores fora do esperado se fizer sentido.

19. Boas práticas de programação específicas para este projeto

19.1 Prefira funções pequenas e nomeadas

Se uma função cresce, extraia para uma função com nome claro.

Ruim:

- uma função gigante com dezenas de responsabilidades.

Bom:

- `build_hydro_fpha_constraints`
- `build_operational_limits`
- `write_cmosist_csv`

19.2 Mantenha a regra de negócio próxima da camada certa

- validação estrutural em `core`
- parsing em `io`
- formulação em `model`
- execução em `simulation`

19.3 Evite números mágicos

Se surgir um fator importante, documente.

Exemplo bom já existente:

- `0.0036 * duracao_horas` para converter `m3/s` em `hm3 por período`

19.4 Nomeie as coisas de forma rastreável

Um bom nome economiza horas de depuração.

Se a variável é por usina, não dê a nome que pareça por unidade.

Se o valor é agregado, deixe explícito.

19.5 Faça análise dimensional sempre

Pergunte:

- o lado esquerdo da restrição está em quã?
- o lado direito está em quã?
- o coeficiente converte ou apenas pondera?

19.6 Valide logo após ler

Quanto mais cedo um erro for pego, melhor.

19.7 Não esconda comportamento importante

Se uma conversão for crítica, deixe o nome claro e, se necessário, documente no código.

19.8 Preserve consistência de padrão

Se o projeto já usa:

- funções `build_...`
- `write_..._csv`
- nomes de variável com índices nomeados

continue no mesmo estilo.

20. Erros comuns ao evoluir o projeto

1. Adicionar parâmetro no CSV e esquecer de colocá-lo no `System`.
 2. Adicionar variável em `variables` e esquecer em `collect_variables`.
 3. Criar restrição mas esquecer de chamá-la em `ConstraintSet::for_system`.
 4. Misturar unidade de entrada com unidade interna.
 5. Repetir valor agregado em linha de unidade e induzir interpretação errada.
 6. Esquecer de tratar flags `UCT`, `UCH` ou `rede`.
 7. Usar nome de restrição ruim e depois não conseguir recuperar dual.
 8. Alterar saída sem atualizar o cabeçalho descritivo.
 9. Fazer mudança estrutural sem adicionar teste manual.
-

21. Fluxo de trabalho recomendado para mudanças reais

Para cada mudança, siga este processo:

1. escreva a matemática;
2. defina a unidade de cada termo;
3. identifique em quais crates a mudança toca;
4. faça a alteração estrutural no `core`, se necessário;
5. ajuste a leitura em `io`;
6. ajuste variáveis, restrições e objetivo em `model`;
7. ajuste solver, se necessário;
8. ajuste simulação e saídas;
9. rode `cargo fmt`;
10. rode `cargo check -p labdessem-cli`;
11. rode testes relevantes;

12. sÃ³ depois valide o caso real.

22. Comandos Ã³teis

Na raiz do projeto:

```
cargo fmt
cargo check -p labdessem-cli
cargo test -p labdessem-model
```

Para localizar rapidamente algo:

```
rg "hydro_fpha" crates
rg "thermal_generation" crates
rg "write_.*csv" crates/labdessem-simulation/src/lib.rs
```

Para rodar o programa:

```
cargo run -p labdessem-cli
```

23. SugestÃ£o prÃ¡tica de leitura para quem vai manter o cÃ³digo

Se vocÃª quiser entender o projeto de forma progressiva, esta ordem ajuda bastante:

1. [crates/labdessem-cli/src/main.rs](#)
2. [crates/labdessem-io/src/study.rs](#)
3. [crates/labdessem-core/src/system.rs](#)
4. [crates/labdessem-model/src/lib.rs](#)
5. [crates/labdessem-model/src/indexing.rs](#)
6. [crates/labdessem-model/src/variables.rs](#)
7. [crates/labdessem-model/src/constraints.rs](#)
8. [crates/labdessem-model/src/objective.rs](#)

9. [crates/labdessem-solver/src/lib.rs](#)

10. [crates/labdessem-simulation/src/lib.rs](#)

24. Fechamento

Se você quiser implementar algo novo sem assistência, a regra de ouro é:

1. defina a matemática;
2. defina as unidades;
3. descubra a camada certa;
4. faça a alteração mínima necessária em cada crate;
5. valide cedo;
6. teste sempre.

Este projeto já está organizado de um jeito que favorece evolução incremental. O segredo é respeitar a separação entre leitura, domínio, formulação, solução e relatório.

Se no futuro você quiser expandir este manual, os melhores próximos tópicos seriam:

- tutorial de implementação de um caso completo de nova variável;
- tutorial de nova restrição de UC;
- tutorial de novo relatório dual;
- checklist de revisão antes de merge;
- padrões de commit e versionamento.